INSTITUTO NACIONAL DE APRENDIZAJE

Núcleo Comercio y Servicios · Subsector Informática y Comunicación

No hay que ser un genio:
hay que tener un método

Taller · De un problema a un programa en C en 8 pasos

Diseño de algoritmos · Módulo CSTI12010

Persona facilitadora

Giovanni Antonio Coto Calderón

LA PREMISA

No hay que ser un genio: hay que tener un método



Siga pasos

Un método convierte un problema grande en decisiones pequeñas y ordenadas.



Divida

Ningún programa se escribe de una vez: se arma con tareas pequeñas que ya sabe hacer.



Hágalo a mano

Si puede resolver un ejemplo con lápiz y calculadora, puede programarlo.

EL MÉTODO

Los 8 pasos

1 Entender el problema

¿Qué me piden con mis palabras?

2 Identificar los datos

¿Qué entra, qué sale, qué no cambia y qué límites hay?

3 Resolver un ejemplo a mano

¿Puedo obtener el resultado con lápiz y calculadora?

4 Dividir en tareas

¿Qué tareas pequeñas forman la solución?

5 Elegir las herramientas

¿Qué instrucción de C resuelve cada tarea y por qué?

6 Comparar formas y elegir

¿De qué maneras se puede programar? ¿Cuál conviene?

7 Algoritmo y prueba de escritorio

¿El algoritmo da el resultado del paso 3?

8 Programar, probar y documentar

¿Compila sin advertencias y pasa todos los casos?

EL PROBLEMA

Venta de entradas de un cine

Un cine registra las compras de una función (de 1 a 10). De cada compra se digita el nombre del cliente (una palabra), el tipo de entrada (1 General 3500, 2 Niño 2500, 3 Adulto mayor 2000) y la cantidad de entradas (de 1 a 8); un dato inválido se vuelve a pedir. Si la compra tiene 5 o más entradas, se aplica un 10 % de descuento. Mostrar una tabla con el cliente, las entradas y el total de cada compra, el total vendido, la compra mayor y cuántas compras tuvieron descuento.

Datos de prueba

- Ana · tipo 1 · 2 entradas
- Luis · tipo 2 · 5 entradas
- Sofia · tipo 3 · 1 entrada
- Mario · tipo 1 · 6 entradas

Inválidos para probar: tipo 4 y 9 entradas.

¿Qué entra, qué sale y qué no cambia?

Paso 1 · Con mis palabras

Se venden entradas a varios clientes; cada compra cuesta según el tipo y la cantidad, con 10 % menos si son 5 o más. Al final se muestra cada compra y un resumen.

Paso 2 · Los datos

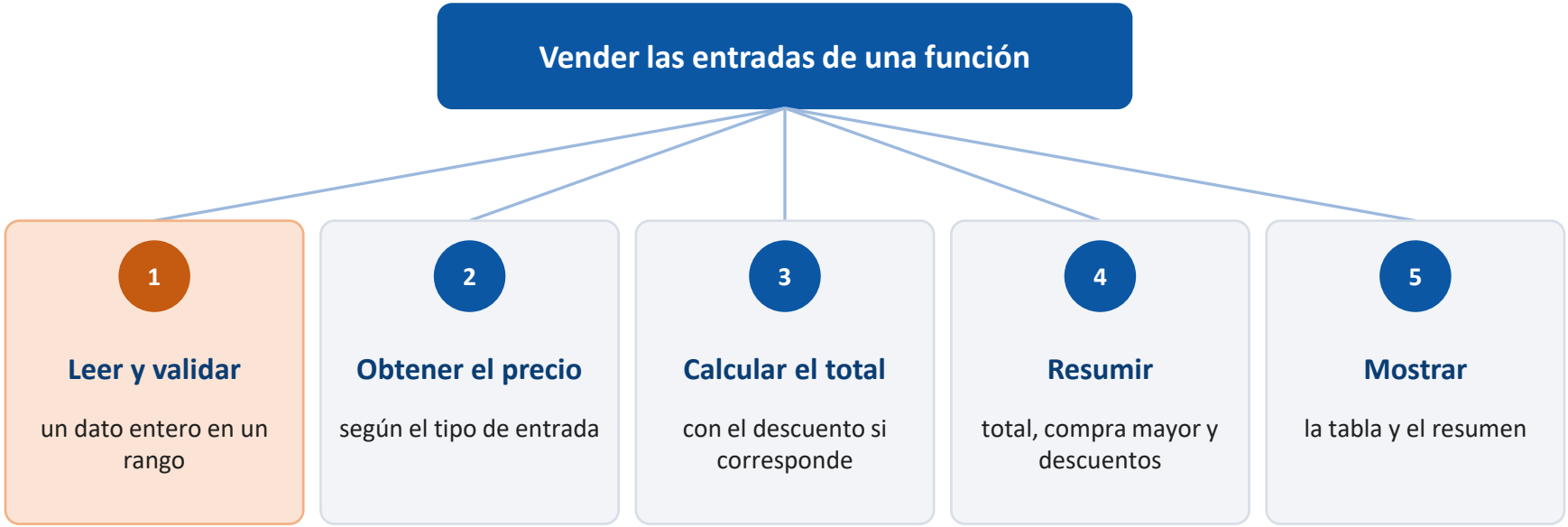
	Qué son
Entradas	Cantidad de compras (1 a 10); por compra: cliente (texto), tipo (1 a 3) y cantidad de entradas (1 a 8).
Salidas	Tabla cliente · entradas · total; total vendido; compra mayor; cantidad de compras con descuento.
Constantes	Precios 3500, 2500 y 2000; 5 entradas para descuento; 10 %; máximo 10 compras.
Restricciones	Datos fuera de rango se vuelven a pedir; el descuento es con 5 o más (no con más de 5).

Si lo hago a mano, lo puedo programar

Cliente	Tipo	Precio	Entradas	Subtotal	¿Descuento?	Total
Ana	1	3500	2	7000	No	**7000**
Luis	2	2500	5	12500	Sí (−1250)	**11250**
Sofia	3	2000	1	2000	No	**2000**
Mario	1	3500	6	21000	Sí (−2100)	**18900**

Resultado esperado: Total vendido 39150 · compra mayor: Mario (18900) · 2 compras con descuento

Un problema grande, cinco tareas pequeñas



La tarea 1 se repite **tres veces** (cantidad, tipo y entradas): es candidata a función.

¿Qué uso, para qué y cómo?

Necesidad	Qué usar	Por qué / para qué	Cómo
Valores que no cambian (precios, 10 %)	**#define**	Un nombre claro; se cambia en un solo lugar.	`#define TASA_DESCUENTO 0.10`
Muchas compras con varios datos	**Arreglos paralelos**	La posición i describe la misma compra.	`char clientes[10][15]; int cantidades[10];`
Nombre del cliente	**Cadena + scanf %14s**	Una palabra sin desbordar el arreglo.	`scanf("%14s", clientes[i]);`
Validar datos (3 veces)	**do-while en una función**	Se escribe una vez y se usa con distintos rangos.	`leerEntero("  Entradas", 1, 8)`
Precio según el tipo (1, 2, 3)	**switch**	Valores fijos: una rama clara por tipo.	`switch (tipo) { case 1: … }`
Descuento con 5 o más	**if**	Es una condición (un rango), no un valor fijo.	`if (cantidad >= MIN_DESCUENTO)`
Recorrer las compras	**for**	La cantidad de vueltas se conoce (n).	`for (i = 0; i < n; i++)`
Tres resultados de una tarea	**Función + punteros**	return devuelve uno; por dirección se devuelven tres.	`resumir(…, &total, &pos, &desc);`

Varias formas correctas, una conveniente

Forma A · Todo en main

if / else if para el precio; tres do-while repetidos; cálculos y resumen dentro de main.

+ Fácil de empezar
+ No necesita funciones
− main largo y repetido
− Difícil de probar por partes

Forma B · Funciones con if

Las mismas funciones, pero el precio se elige con if / else if y el resumen con tres funciones que recorren el arreglo.

+ Ordenada y reutilizable
+ if sirve para cualquier condición
− if largo para valores fijos
− Tres recorridos del arreglo

Forma C · Funciones + switch + punteros

leerEntero, precioEntrada (switch), calcularTotal y resumir, que devuelve tres resultados por dirección en un solo recorrido.

+ main se lee como un índice
+ Una sola pasada por los datos
+ Cada regla en un solo lugar
− Requiere entender punteros

ELEGIDA

Criterios:

- Se entiende al leerlo
- No repite código
- Cada regla en un solo lugar
- Se prueba por partes
- Hace el menor trabajo

¿Cuándo switch y cuándo if?

Con switch (elegido)

cine.c (sin comentarios)

```
58  double precioEntrada(int tipo) {
59      switch (tipo) {
60          case 1:
61              return PRECIO_GENERAL;
62          case 2:
63              return PRECIO_NINO;
64          default:
65              return PRECIO_MAYOR;
66      }
67  }
```

switch: una variable comparada con **valores fijos** (tipo 1, 2, 3; plan; zona; opción de menú).

Con if / else if (también válido)

alternativa

```
double precioEntrada(int tipo) {
    if (tipo == 1) {
        return PRECIO_GENERAL;
    } else if (tipo == 2) {
        return PRECIO_NINO;
    } else {
        return PRECIO_MAYOR;
    }
}
```

if: rangos o varias condiciones (cantidad >= 5, edad >= 65, peso > 20). switch no puede evaluar rangos.

Antes de programar, se prueba en papel

Pseudocódigo (paso 7)

```
Inicio
  n <- leerEntero("Cantidad de compras", 1, 10)
  Para i <- 0 hasta n - 1
    Leer clientes[i]
    tipo <- leerEntero("Tipo", 1, 3)
    cantidades[i] <- leerEntero("Entradas", 1, 8)
    totales[i] <- precio(tipo) * cantidades[i]
    Si cantidades[i] >= 5: totales[i] <- totales[i] * 0.9
  FinPara
  resumir -> totalVendido, posMayor, conDescuento
  Mostrar tabla y resumen
Fin
```

Prueba de escritorio de resumir

i	tot[i]	cant[i]	*total	*posMayor	*conDescuento
—	—	—	0	0	0
0	7000	2	7000	0	0
1	11250	5	18250	1	1
2	2000	1	20250	1	1
3	18900	6	39150	3	2

Final: **39150**, posición **3** y **2** descuentos = lo calculado a mano en el paso 3.

main organiza el trabajo

cine.c sin comentarios · líneas 26 a 37

```
26      n = leerEntero("Cantidad de compras", 1, MAX_COMPRAS);
27
28      for (i = 0; i < n; i++) {
29          printf("Compra %d\n  Cliente: ", i + 1);
30          scanf("%14s", clientes[i]);
31          tipo = leerEntero("  Tipo", 1, 3);
32          cantidades[i] = leerEntero("  Entradas", 1, 8);
33          totales[i] = calcularTotal(tipo, cantidades[i]);
34      }
35
36      resumir(cantidades, totales, n, &totalVendido, &posMayor,
37              &conDescuento);
```

main no calcula nada: **llama** a las funciones en el orden del algoritmo. Se lee como un índice.

Leer, elegir el precio y calcular

cine.c sin comentarios · líneas 49 a 74 (precioEntrada se vio con el switch)

```
49  int leerEntero(const char mensaje[], int minimo, int maximo) {
50      int valor;
51      do {
52          printf("%s (%d a %d): ", mensaje, minimo, maximo);
53          scanf("%d", &valor);
54      } while (valor < minimo || valor > maximo);
55      return valor;
56  }
57
        …
69  double calcularTotal(int tipo, int cantidad) {
70      double total = precioEntrada(tipo) * cantidad;
71      if (cantidad >= MIN_DESCUENTO)
72          total = total * (1 - TASA_DESCUENTO);
73      return total;
74  }
```

Una función por regla: si mañana cambia el **descuento**, solo se toca calcularTotal.

resumir: tres resultados con punteros

cine.c sin comentarios · líneas 76 a 89

```
76  void resumir(const int cant[], const double tot[], int n,
77               double *total, int *posMayor, int *conDescuento) {
78      int i;
79      *total = 0;
80      *posMayor = 0;
81      *conDescuento = 0;
82      for (i = 0; i < n; i++) {
83          *total += tot[i];
84          if (tot[i] > tot[*posMayor])
85              *posMayor = i;
86          if (cant[i] >= MIN_DESCUENTO)
87              (*conDescuento)++;
88      }
89  }
```

return devuelve **un** valor. Con punteros, resumir escribe **tres** resultados en las variables de main en una sola pasada.

El archivo cine.c completo trae un comentario QUÉ / CÓMO / PARA QUÉ antes de cada línea.

¿Da lo mismo que a mano?

Salida real (gcc)

```
Compra 2
  Cliente: Luis
  Tipo (1 a 3): 2
  Entradas (1 a 8): 9
  Entradas (1 a 8): 5
…
Cliente      Entradas       Total
Ana                 2     7000.00
Luis                5    11250.00
Sofia               1     2000.00
Mario               6    18900.00
Total vendido:           39150.00
Compra mayor:        Mario (18900.00)
Con descuento:       2 compra(s)
```

Caso	Datos	Resultado
Normal	Ana, tipo 1, 2 entradas	7000.00 sin descuento
Límite	Luis, tipo 2, exactamente 5 entradas	11250.00 (sí hay descuento: >= 5)
Inválido	Tipo 4 y 9 entradas	Se vuelven a pedir
Resumen	Las 4 compras	39150.00 · Mario · 2 con descuento

Documentar: encabezado del .c (enunciado, qué, cómo, resultado) y README con los 8 pasos.

AHORA USTEDES

Réplica en grupos de 3 o 4 personas

1 **Cobro diario de un parqueo**

`parqueo.c`

2 **Facturación de datos móviles**

`datos.c`

3 **Cobro de envíos de paquetes**

`envios.c`

4 **Inscripciones de un gimnasio**

`gimnasio.c`

Cada grupo

- Aplica los 8 pasos en la hoja de trabajo.
- Paso 6: menciona al menos 2 formas, elige 1 con criterios.
- Programa SOLO la forma elegida.
- Prueba: normal, límite e inválido.
- Presenta en 5 minutos.

Roles (rotan en cada paso)

Lector
lee y reescribe

Analista
datos y tareas

Programador
escribe el código

Probador
pruebas y README